

PROYECTO FINAL DE GRADO – COSTURAS CON MIMO WEB VUE/DJANGO/POSTGRES – MEMORIA



Alumno: Pablo Díaz Diez

Profesor: Jose María Herrero

Fecha: 07/04/2026

2º Desarrollo de Aplicaciones Multiplataforma

1. Índice

1. Índice	2
2. Índice de ilustraciones.....	4
3. Memoria descriptiva	5
Introducción.....	5
Abstract	5
Palabras clave	5
4. Objetivos del proyecto	5
5. Lenguajes empleados.....	6
Vue.js.....	6
Django	7
PostgreSQL.....	7
Servicios externos:.....	7
Stripe	7
Mailtrap	7
6. Recursos	8
Hardware	8
Software	8
Humanos	8
Previsión económica.....	9
7. Descripción completa	9
8. Base de datos(E/R)	10
9. Diagrama de casos de uso	12
10. Diagrama de clases.....	15
11. Diagrama de estados	16
12. Diagrama de secuencia	19
Diagrama secuencia: Encargo	19
Diagrama secuencia: Login.....	19
Diagrama secuencia: Pago	20
13. Diagrama de paquetes	21
14. Diagrama de objetos	23

15.	Valoraciones	24
16.	Autoevaluación.....	24
17.	Conclusiones	25
18.	Bibliografía/Webgrafía	26

2. Índice de ilustraciones

Ilustración 1 Diagrama E/R.....	11
Ilustración 2 Caso de uso Cliente	12
Ilustración 3 Caso de uso Admin.....	14
Ilustración 4 Diagrama de Clases	15
Ilustración 5 Diagrama de estados Encargos	17
Ilustración 6 Diagrama de estados Login	18
Ilustración 7 Diagrama de secuencia: Encargo	19
Ilustración 8 Diagrama de secuencia: Login	20
Ilustración 9 Diagrama de secuencia: Pagos.....	21
Ilustración 10 Diagrama de paquetes.....	22
Ilustración 11 Diagrama de objetos	23

3. Memoria descriptiva

Introducción

Este Trabajo de Fin de Grado presenta el diseño, desarrollo e implementación de "Costuras con Mimo", una plataforma web de comercio electrónico (e-commerce) orientada a la venta de artículos de costura artesanal. A diferencia de las tiendas online tradicionales, este sistema permite un alto grado de personalización en los pedidos, ofreciendo a los clientes la posibilidad de seleccionar tejidos específicos y añadir detalles únicos, como bordados con nombre.

Abstract

This Final Degree Project presents the design, development and implementation of "Costuras con Mimo", a web-based e-commerce platform focused on the sale of handmade sewing articles. Unlike traditional online stores, this system allows a high degree of customisation in orders, offering customers the possibility of selecting specific fabrics and adding unique details, such as embroidery with a name.

Palabras clave

Personalización, Frontend, Backend, Stripe, Dashboard

4. Objetivos del proyecto

El objetivo principal de este proyecto es desarrollar una plataforma de comercio electrónico funcional y lista para su salida al mercado, que digitalice y automatice la gestión de un negocio de artesanía textil.

Para alcanzar dicho objetivo, se han definido los siguientes objetivos específicos:

Objetivos funcionales:

- Sistema de encargos personalizados: Implementar un motor de personalización que permita al cliente seleccionar un producto base, elegir el tejido entre el muestrario disponible en el taller y añadir un texto de bordado opcional.
- Pasarela de pago: Integrar un sistema de cobro seguro mediante Stripe, que permita procesar pagos con tarjeta sin almacenar datos bancarios en los servidores propios de la aplicación.
- Gestión de pedidos: Desarrollar un sistema de seguimiento de pedidos que permita a la administradora actualizar el estado de cada pedido y notifique automáticamente al cliente por correo electrónico ante cualquier cambio.

- Gestión de stock automática: Implementar un sistema que oculte automáticamente del catálogo los productos cuyo stock llegue a cero.
- Notificaciones automáticas por email: Integrar un servicio de envío de correos transaccionales que informe al cliente del estado de sus pedidos en tiempo real.
- Sistema de favoritos: Permitir a los usuarios marcar productos como favoritos para facilitar futuras compras y proporcionar información relevante sobre la demanda del catálogo. Este objetivo queda pendiente de implementación para una versión futura de la aplicación.

Objetivos técnicos:

- Arquitectura desacoplada: Desarrollar el proyecto bajo una arquitectura cliente-servidor separada, con Vue.js en el frontend y Django REST Framework en el backend, comunicados mediante una API REST.
- Diseño responsive: Garantizar que la interfaz sea completamente funcional tanto en dispositivos móviles como en ordenadores de escritorio.
- Doble vista según rol: Implementar dos entornos diferenciados: una tienda pública para el cliente, con catálogo, filtros y buscador, y un panel de administración privado para la gestión del negocio.
- Seguridad en las rutas: Proteger las rutas privadas mediante un sistema de guardián en el router de Vue que verifique el token de autenticación y el rol del usuario antes de permitir el acceso.
- Organización y buenas prácticas: Estructurar el código siguiendo las convenciones recomendadas de cada tecnología, separando responsabilidades entre componentes, vistas, servicios y modelos.
- Pantallas de error personalizadas: Diseñar páginas de error 404 y 403 acordes a la identidad visual de la aplicación, mejorando la experiencia de usuario ante situaciones inesperadas.

5. Lenguajes empleados

Vue.js

Vue.js 3 es el framework de JavaScript utilizado para el desarrollo del frontend. Permite construir interfaces de usuario mediante componentes reutilizables, lo que favorece una organización modular y mantenible del código. Al tratarse de una Single Page Application (SPA), la navegación entre secciones es fluida y no requiere recargas completas de la página. El sistema de enrutamiento se gestiona a través de Vue Router, que permite definir las rutas de la aplicación y proteger el acceso a determinadas vistas según el rol del usuario autenticado. Para la construcción y compilación del proyecto se

utiliza Vite como herramienta de desarrollo, que proporciona un entorno ágil con recarga en caliente.

Django

Django REST Framework es la biblioteca utilizada para el desarrollo del backend. Proporciona una serie de herramientas y funcionalidades que facilitan la creación de APIs web robustas y escalables. En este proyecto, Django no se utiliza para renderizar plantillas HTML, sino exclusivamente como API: recibe las peticiones del frontend, procesa la lógica de negocio y devuelve los datos en formato JSON. Se ha elegido esta tecnología principalmente por el sistema de autenticación y gestión de permisos que incorpora de serie, así como por la solidez y madurez del framework.

PostgreSQL

PostgreSQL es el sistema gestor de bases de datos relacional utilizado en el proyecto. Se ha optado por esta tecnología por su fiabilidad, su soporte completo de transacciones y su capacidad para gestionar relaciones complejas entre tablas mediante claves foráneas, aspectos fundamentales en un sistema que maneja datos críticos como transacciones económicas y control de stock. La comunicación entre Django y PostgreSQL se realiza a través del ORM propio del framework, que abstrae las consultas SQL y garantiza la integridad de los datos.

Servicios externos:

Stripe

Stripe es la pasarela de pagos integrada en la aplicación para gestionar los cobros con tarjeta de forma segura. Su principal ventaja es que el proceso de pago se realiza íntegramente en los servidores de Stripe, de modo que los datos bancarios de los clientes nunca llegan a almacenarse en la base de datos propia de la aplicación. La integración se ha realizado mediante su API oficial, que proporciona herramientas de prueba que permiten simular transacciones sin realizar cargos reales durante la fase de desarrollo.

Mailtrap

Mailtrap es el servicio externo utilizado para gestionar el envío de correos electrónicos durante la fase de desarrollo. Funciona como un servidor SMTP seguro que intercepta todos los emails generados por la aplicación (confirmaciones de pedido, actualizaciones de estado, etc.), permitiendo verificar su diseño y funcionamiento sin riesgo de enviar

comunicaciones a direcciones reales. Su integración con el backend de Django se realiza a través de los parámetros SMTP definidos en el archivo settings.py.

6. Recursos

Hardware

Para el desarrollo del proyecto se ha utilizado el siguiente equipamiento físico:

- Ordenador de sobremesa con procesador Intel Core i5, 32GB de RAM y 1TB SSD, utilizado como máquina principal de desarrollo.
- Monitor secundario para facilitar el trabajo simultáneo entre el editor de código y el navegador.
- Extensión de navegador para las pruebas de responsive design y verificación del comportamiento de la web en dispositivos móviles.

Para la distribución, al tratarse de una aplicación web, el único requisito del cliente es disponer de un dispositivo con conexión a internet y un navegador actualizado.

Software

Las herramientas de software utilizadas durante el desarrollo han sido:

- Visual Studio Code como editor principal de código.
- pgAdmin 4 para la administración visual de la base de datos PostgreSQL.
- Git para el control de versiones del proyecto.
- Google Chrome y sus herramientas de desarrollador para las pruebas y depuración.
- draw.io para la elaboración de los diagramas UML.

Humanos

El proyecto ha sido desarrollado íntegramente por una sola persona, el alumno Pablo Díaz Diez, asumiendo los roles de analista, diseñador y programador. Se estima una dedicación aproximada de 180 horas distribuidas de la siguiente manera:

- Análisis y diseño: 30 horas
- Desarrollo del backend (Django): 50 horas

- Desarrollo del frontend (Vue.js): 60 horas
- Integración de servicios externos (Stripe, Mailtrap): 20 horas
- Pruebas y corrección de errores: 20 horas

Previsión económica

Aunque el proyecto tiene un carácter académico y familiar, su puesta en producción y viabilidad comercial conllevan los siguientes costes estimados:

- Costes de infraestructura (Fijos): Registro del dominio web (aprox. 15€/año) y alquiler de un servidor VPS para alojar la aplicación y la base de datos PostgreSQL (aprox. 60€/año).
- Costes operativos (Variables): La pasarela de pago integrada (Stripe) no tiene cuota de mantenimiento fijo, pero aplica una comisión por cada transacción exitosa (aproximadamente 1.5% + 0.25€ por pago con tarjeta europea).
- Costes de desarrollo: Tomando como referencia las 180 horas estimadas en el apartado de recursos humanos, a un precio de mercado junior de 20€/hora, el desarrollo del software estaría valorado en unos 3.600€. Al tratarse de un Trabajo Fin de Grado y un proyecto familiar, este coste queda amortizado desde el primer día.

7. Descripción completa

La plataforma web "Costuras con Mimo" se estructura en dos entornos diferenciados según el rol del usuario autenticado: el entorno del cliente y el entorno de administración.

Entorno del cliente

Al acceder a la aplicación, el usuario encuentra el catálogo de productos disponibles, donde puede visualizar cada artículo con su imagen, precio y categoría. Dispone de una barra de herramientas con filtros por categoría y artesano, un buscador por nombre y opciones de ordenación por precio y fecha. Al acceder a la ficha de un producto, puede consultar su descripción completa, la disponibilidad en stock y añadirlo al carrito de compra.

Desde el menú de navegación, el cliente puede acceder al formulario de encargo personalizado, donde selecciona el tipo de producto que desea, elige el tejido entre el muestrario visual disponible en el taller y puede añadir un texto de bordado opcional. Una vez configurado, el encargo se añade al carrito con todos sus detalles.

El carrito de compra muestra el listado de artículos seleccionados, permitiendo modificar las cantidades o eliminar productos. Al confirmar la compra, el sistema redirige al cliente a la pasarela de pago segura de Stripe, donde introduce los datos de su tarjeta. Si el

pago es aceptado, el sistema crea el pedido automáticamente, descuenta el stock correspondiente y muestra una pantalla de confirmación.

El área personal del cliente le permite consultar sus datos de registro, modificar su dirección de envío y contraseña, y revisar el historial completo de sus pedidos con su estado actualizado en tiempo real. Cada vez que la administradora actualiza el estado de un pedido, el cliente recibe automáticamente una notificación por correo electrónico.

La aplicación cuenta además con una página de contacto donde se detalla la filosofía del negocio y un enlace directo al perfil de Instagram del taller. Para mejorar la experiencia de usuario, se han implementado páginas de error personalizadas: una página 404 para URLs inexistentes y una página 403 para intentos de acceso no autorizado al panel de administración.

Entorno de administración

El acceso al panel de administración está restringido a usuarios con permisos de administrador. Al iniciar sesión, el sistema detecta automáticamente el rol del usuario y le redirige al dashboard de gestión.

El dashboard se divide en dos áreas principales. En la columna izquierda se encuentra el inventario, donde la administradora puede gestionar productos, categorías y tejidos: crear nuevos registros, editar el precio, el stock o la imagen de los existentes, y eliminarlos cuando sea necesario. En la columna derecha se muestran los últimos pedidos recibidos, donde puede buscar por número de pedido o nombre de cliente, consultar el recibo detallado de cada compra y actualizar el estado de cada pedido mediante un desplegable. Cada cambio de estado genera automáticamente un correo de notificación al cliente.

En la barra inferior del dashboard se muestra un resumen estadístico del negocio: el número total de telas activas en el catálogo, el número total de pedidos históricos y la facturación total acumulada.

8. Base de datos(E/R)

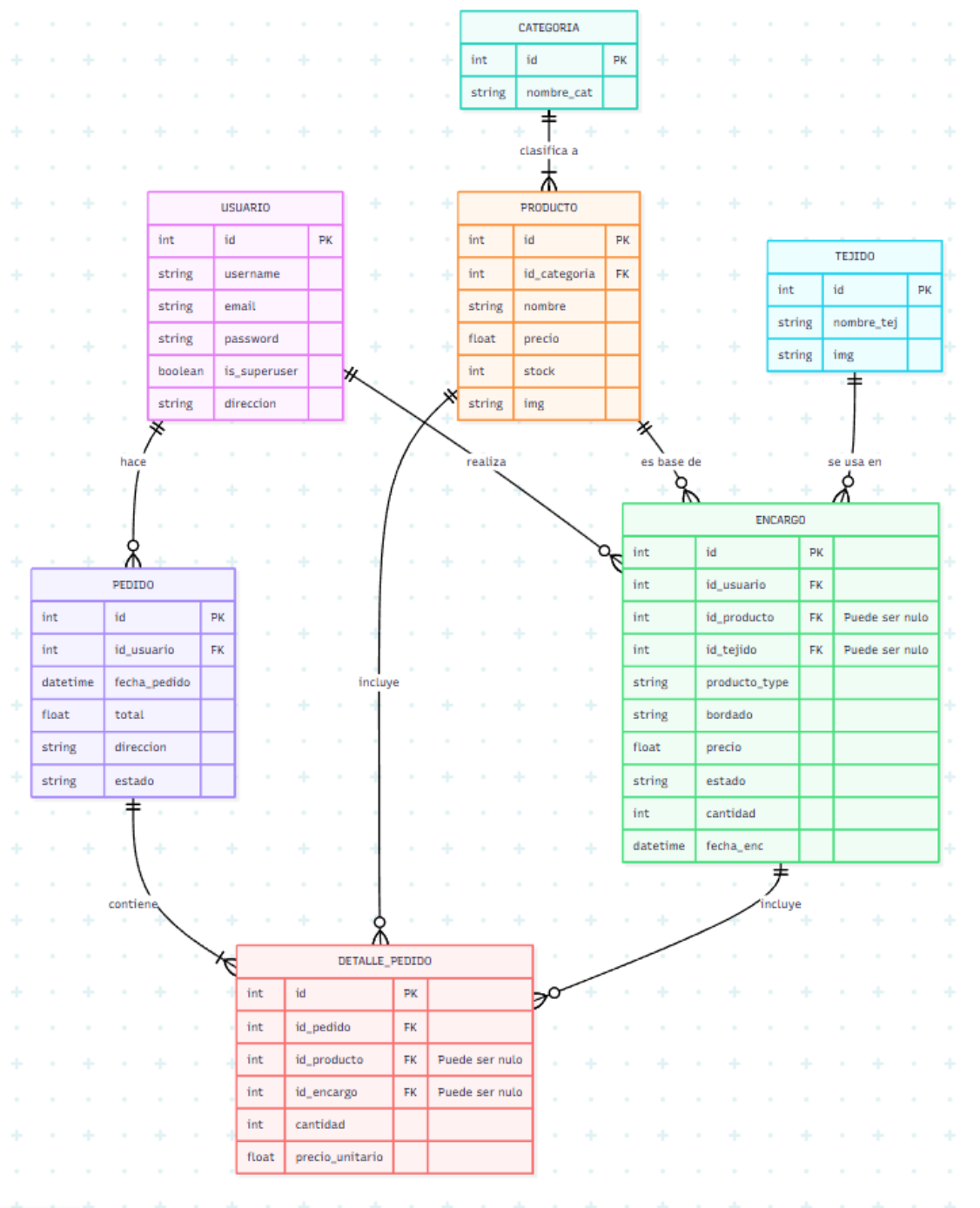


Ilustración 1 Diagrama E/R

El esquema de la base de datos está compuesto por siete tablas relacionadas entre sí mediante claves foráneas, garantizando la integridad referencial de todos los datos del sistema.

La tabla **Usuario** almacena las credenciales y datos personales de cada cuenta registrada, incluyendo el campo `is_staff`, que diferencia a los administradores de los clientes y determina el nivel de acceso al sistema.

Las tablas **Categoría** y **Tejido** actúan como catálogos auxiliares. La primera permite clasificar los productos en familias (como "Bebé" o "Hogar"), mientras que la segunda almacena el muestrario de telas disponibles en el taller junto con su imagen. Ambas se relacionan directamente con la tabla **Producto**, que registra el nombre, precio, imagen y stock de cada artículo. Cuando el stock de un producto llega a cero, el sistema lo oculta automáticamente del catálogo.

La gestión de ventas se divide en dos tablas complementarias. La tabla **Pedido** almacena la cabecera de cada compra: el usuario que la realiza, la fecha, el importe total, el estado y la dirección de envío. La tabla **DetallePedido** desglosa cada pedido en sus líneas individuales, guardando el precio unitario en el momento exacto de la compra para evitar inconsistencias si el precio del producto se modifica posteriormente.

Finalmente, la tabla **Encargo** gestiona las personalizaciones, registrando el tipo de producto solicitado, el tejido elegido, el texto a bordar y el estado de fabricación de cada encargo de forma independiente.

9. Diagrama de casos de uso

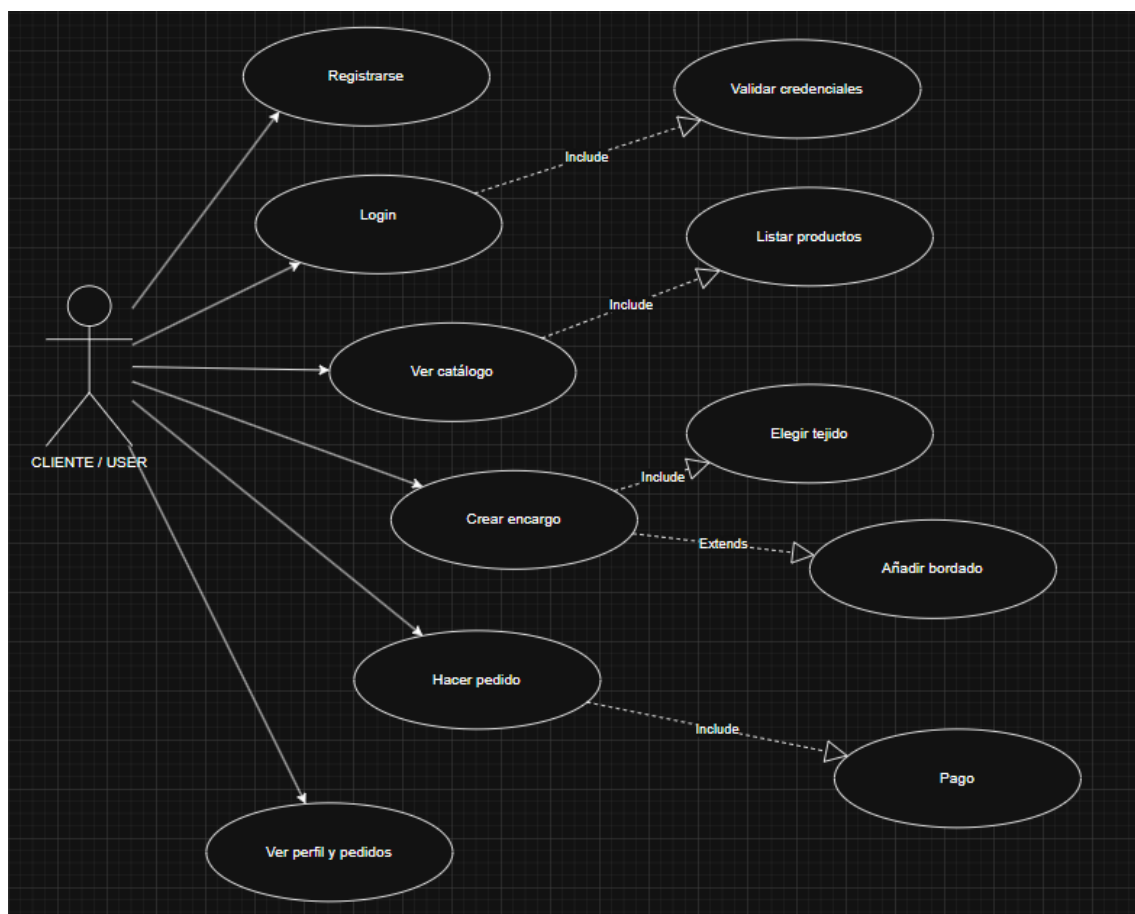


Ilustración 2 Caso de uso Cliente

Este diagrama representa las funcionalidades disponibles para el Cliente/Usuario de la plataforma. El punto de entrada al sistema es el proceso de **Login**, que incluye obligatoriamente (<<include>>) la **Validación de credenciales** contra la base de datos para verificar la identidad del usuario.

Una vez autenticado, el usuario tiene acceso a tres áreas funcionales principales. La primera es el catálogo de la tienda: el usuario puede **Ver el catálogo**, acción que internamente ejecuta **Listar productos** para mostrar los artículos disponibles. Desde la ficha de producto, puede **Hacer un pedido**, proceso que incluye obligatoriamente (<<include>>) el paso por la **Pasarela de pago** para confirmar la compra.

La segunda área es la personalización mediante el sistema de **Encargo**. Al configurar su producto, el cliente debe **Elegir la tela** de forma obligatoria (<<include>>), y tiene la opción de **Añadir Bordado** de forma opcional (<<extend>>), pudiendo decidir si desea personalizar el producto con un texto bordado o no.

La tercera área es la **Gestión de Favoritos**. El usuario puede **Listar sus favoritos**, con la posibilidad de **Filtrar por favoritos** para localizar artículos rápidamente. Desde la ficha de producto o su área personal, puede **Añadir** o **Eliminar favoritos** según su preferencia.

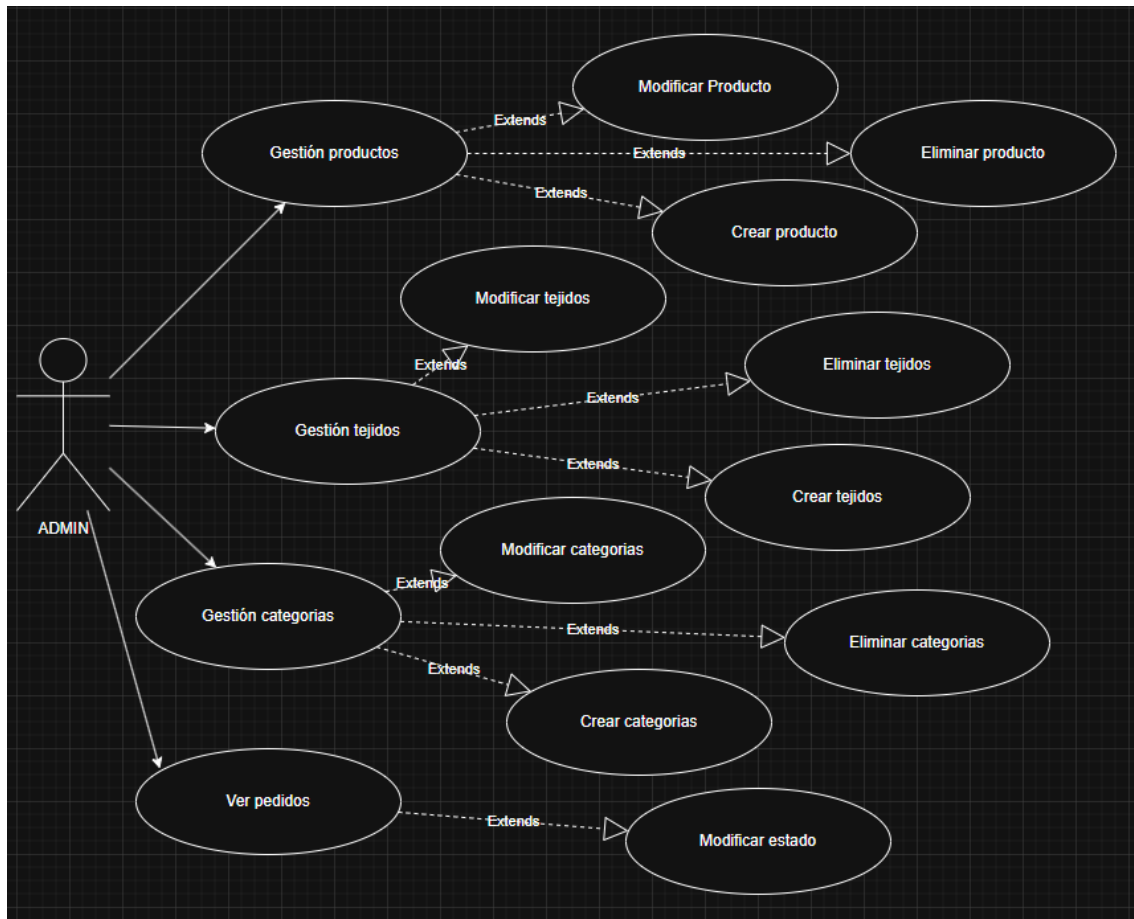


Ilustración 3 Caso de uso Admin

Este diagrama representa las funcionalidades disponibles para el **Administrador** del sistema, encargado de gestionar el negocio a través del panel de control.

El administrador interactúa con cuatro bloques funcionales principales. El primero es la gestión del catálogo. El caso de uso **Crear producto** incluye obligatoriamente (<<include>>) **Crear categoría**, dado que todo producto debe estar asociado a una categoría existente en el sistema antes de poder ser registrado. Una vez creado el producto, el administrador puede **Modificar producto** de forma opcional (<<extend>>), actuando sobre registros ya existentes cuando sea necesario realizar cambios en el precio, la imagen o el stock.

Los otros tres bloques son independientes entre sí. El administrador puede **Crear tejido** para añadir nuevas telas al muestrario disponible para los encargos personalizados. También puede acceder a **Ver pedidos** para consultar el listado de compras realizadas. Desde esta vista, tiene la opción de **Cambiar estado** de cada pedido de forma opcional (<<extend>>), actualizando el progreso de fabricación y envío cuando sea necesario, lo que genera automáticamente una notificación por correo al cliente.

10. Diagrama de clases

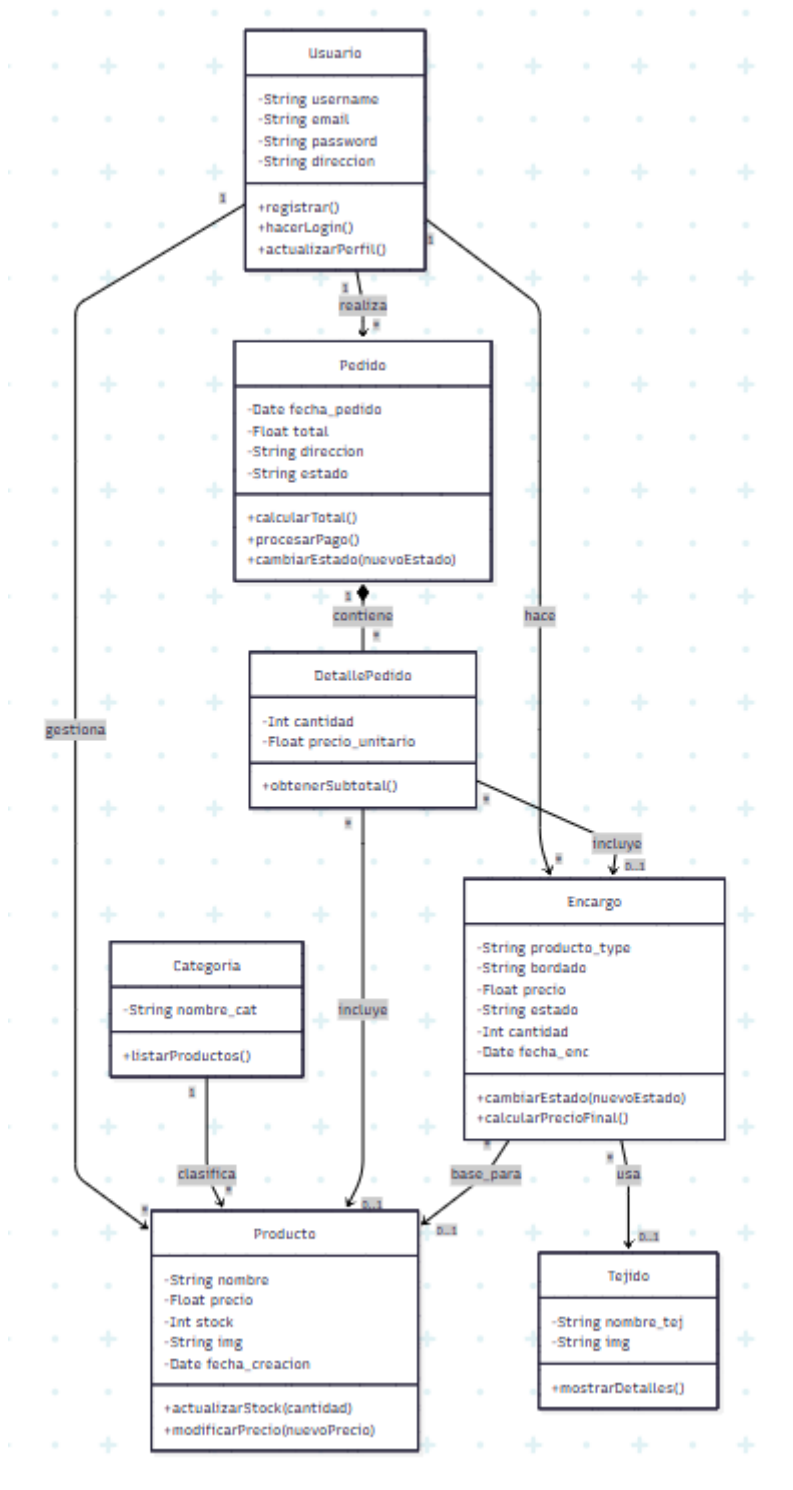


Ilustración 4 Diagrama de Clases

El diagrama de clases representa la estructura estática del sistema, mostrando las entidades principales de la aplicación, sus atributos, métodos y las relaciones entre ellas.

El sistema se compone de siete clases principales. La clase **Usuario** centraliza la gestión de accesos, almacenando las credenciales y datos personales, y expone métodos para el registro, autenticación y actualización del perfil. La clase **Categoria** permite clasificar los productos del catálogo, ofreciendo un método para listar todos los productos asociados a ella.

La clase **Producto** almacena la información del catálogo base (nombre, precio, stock e imagen) y dispone de métodos para actualizar el stock y modificar el precio. Se relaciona con **Categoria** mediante una clave foránea. La clase **Tejido** guarda el muestrario de telas disponibles en el taller, con su nombre e imagen.

La clase **Encargo** es la pieza central de la personalización, vinculando un producto base con un tejido específico y un texto de bordado. Incluye métodos para cambiar su estado a lo largo del proceso de fabricación y para calcular el precio final. La clase **Pedido** registra la cabecera de cada compra, con métodos para calcular el total, procesar el pago y gestionar su estado. Finalmente, la clase **DetallePedido** actúa como línea de detalle de cada pedido, vinculando cada artículo (producto o encargo) con su cantidad y precio unitario en el momento de la compra.

11. Diagrama de estados

Este diagrama representa el ciclo de vida completo de un encargo personalizado dentro del sistema. El estado inicial es **CARRITO**, que es cuando el cliente ha configurado y añadido su encargo personalizado pero aún no ha confirmado el pago. Una vez realizado el pago correctamente, el encargo pasa a estado **PROCESADO**, indicando que el cobro se ha completado y está pendiente de que la administradora lo acepte. Cuando la artesana se pone a trabajar en él, cambia el estado a **EN_PREPARACION**. Una vez terminado y enviado por correo, pasa a **ENVIADO**, y finalmente a **ENTREGADO** cuando el cliente lo recibe. En cualquier momento entre **PROCESADO** y **EN_PREPARACION**, la administradora puede cambiar el estado a **CANCELADO** si hubiera algún problema con el encargo.

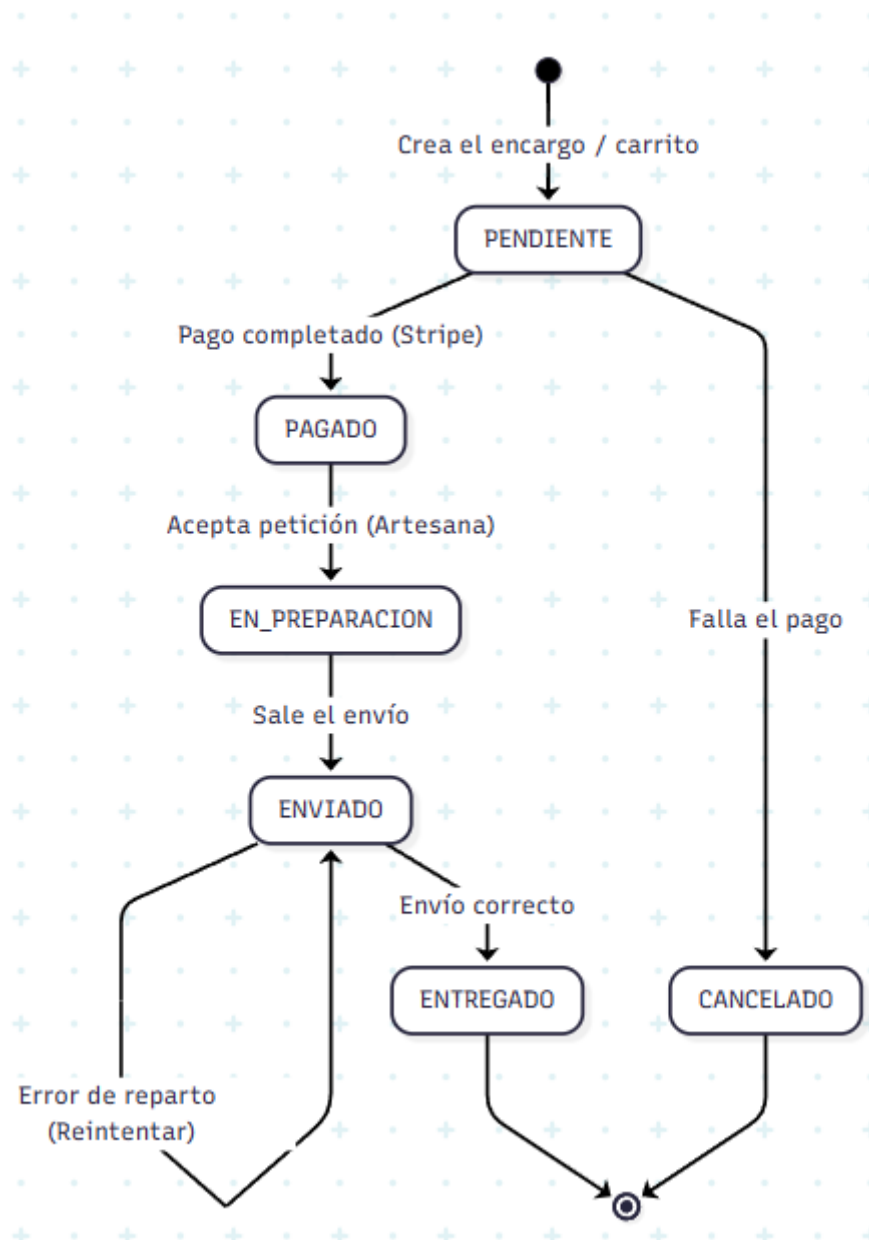


Ilustración 5 Diagrama de estados Encargos

Este diagrama muestra los posibles estados de sesión de un usuario en la aplicación. El estado inicial es DESCONECTADO, que es el estado por defecto de cualquier visitante. Cuando el usuario introduce sus credenciales, el sistema pasa al estado VALIDANDO, donde Django comprueba el usuario y la contraseña contra la base de datos. Si las credenciales son incorrectas, el sistema vuelve al estado DESCONECTADO mostrando un error. Si son correctas, se genera un token JWT y el sistema pasa a CONECTADO. Desde ahí, el sistema comprueba el campo `is_staff` del usuario: si es `true`, se redirige al usuario al panel de ADMIN con acceso al dashboard; si es `false`, accede como CLIENTE a la tienda. En ambos casos, al cerrar sesión se elimina el token y se vuelve al estado DESCONECTADO.

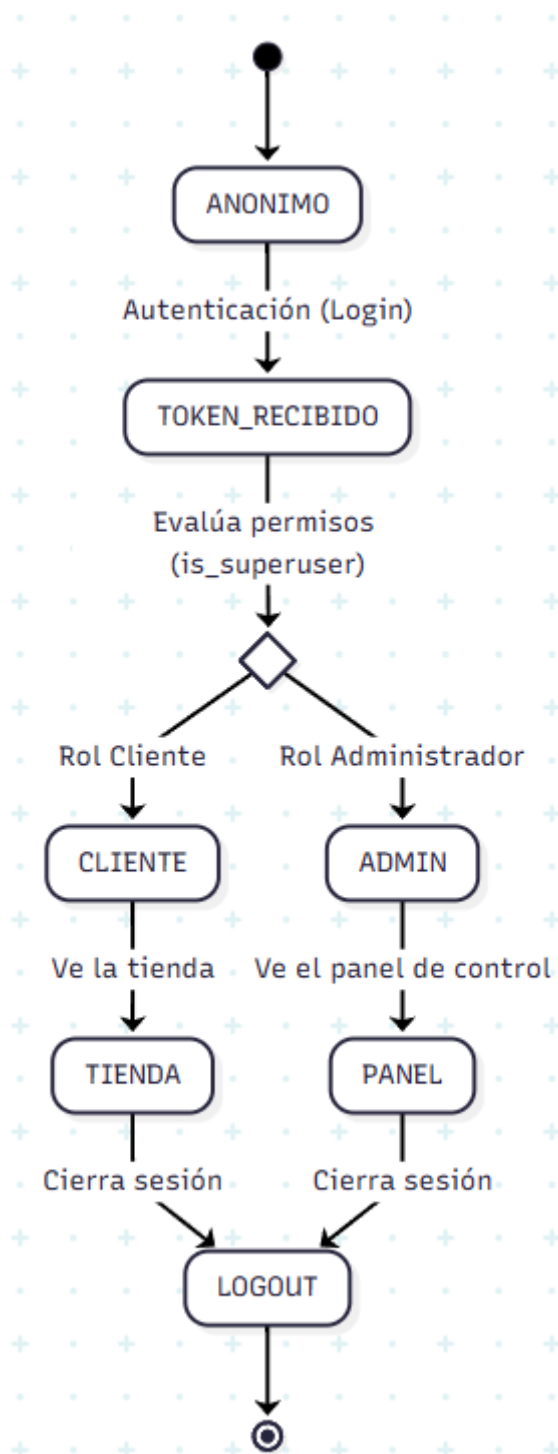


Ilustración 6 Diagrama de estados Login

12. Diagrama de secuencia

Diagrama secuencia: Encargo

Este diagrama representa el flujo principal de la aplicación: la creación de un encargo personalizado. El cliente introduce los datos del encargo (tejido, bordado, etc.) en el formulario y pulsa "Confirmar Encargo". La interfaz Vue envía una solicitud POST al controlador Django mediante Axios. El serializador valida la estructura e integridad de los datos recibidos. Si los datos son válidos, el modelo guarda el nuevo registro en PostgreSQL y devuelve una confirmación, mostrando al cliente el mensaje "Pedido Enviado". Si los datos no son válidos, el serializador devuelve un error de validación con una respuesta HTTP 400 y la interfaz muestra el mensaje de error al usuario.

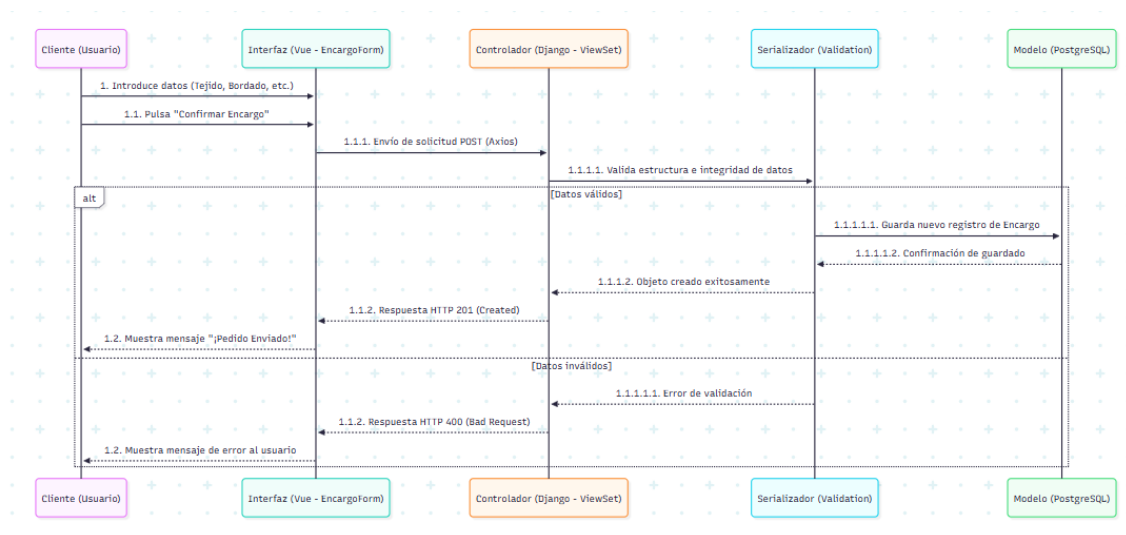


Ilustración 7 Diagrama de secuencia: Encargo

Diagrama secuencia: Login

Este diagrama muestra el proceso de autenticación de un usuario. El cliente introduce su usuario y contraseña en el modal de login y pulsa "Entrar". La interfaz envía una solicitud POST al controlador Django, que comprueba las credenciales contra la base de datos. Si son correctas, el servidor devuelve una respuesta HTTP 200 junto con un token JWT, que la interfaz guarda en el localStorage y redirige al usuario a la tienda. Si las credenciales son incorrectas, el servidor responde con un HTTP 401 y la interfaz muestra un mensaje de error al usuario.

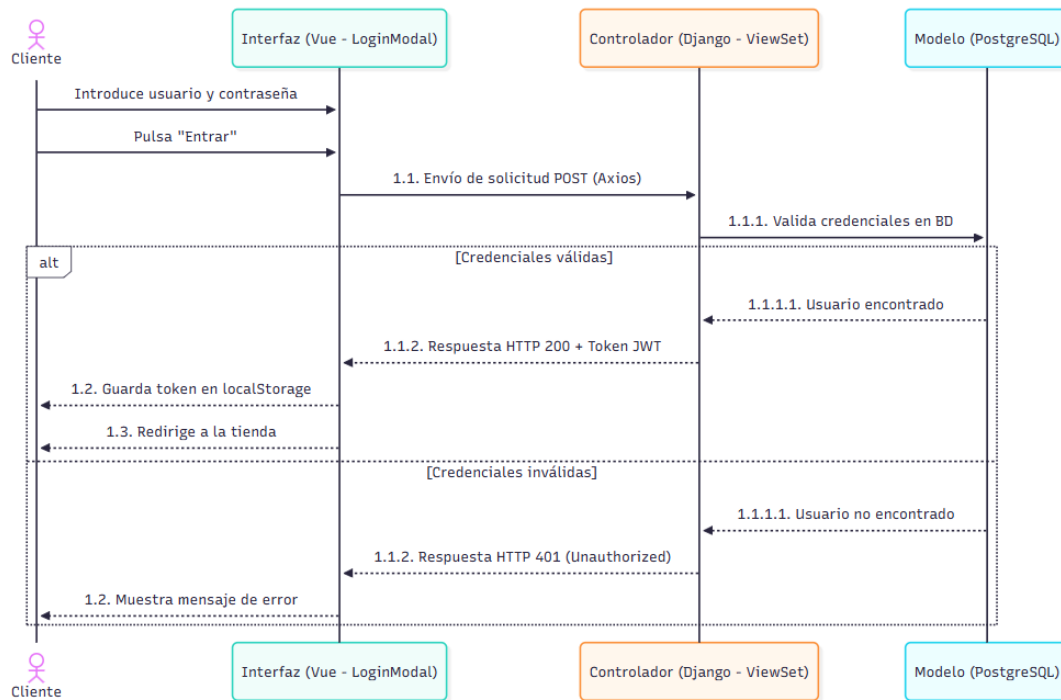


Ilustración 8 Diagrama de secuencia: Login

Diagrama secuencia: Pago

Este diagrama representa el flujo completo de una compra. El cliente pulsa "Pagar Pedido" desde el carrito, lo que lanza una solicitud POST al controlador Django. Este crea una sesión de pago en Stripe, que devuelve una URL de pago. El sistema redirige al cliente a la pasarela de Stripe, donde introduce los datos de su tarjeta. Si el pago es aceptado, Stripe notifica al controlador, que crea el pedido en la base de datos, descuenta el stock de los productos comprados y devuelve una respuesta satisfactoria, mostrando al cliente la pantalla de "¡Pago Exitoso!". Si el pago es rechazado, Stripe devuelve un error y la interfaz informa al cliente del problema.

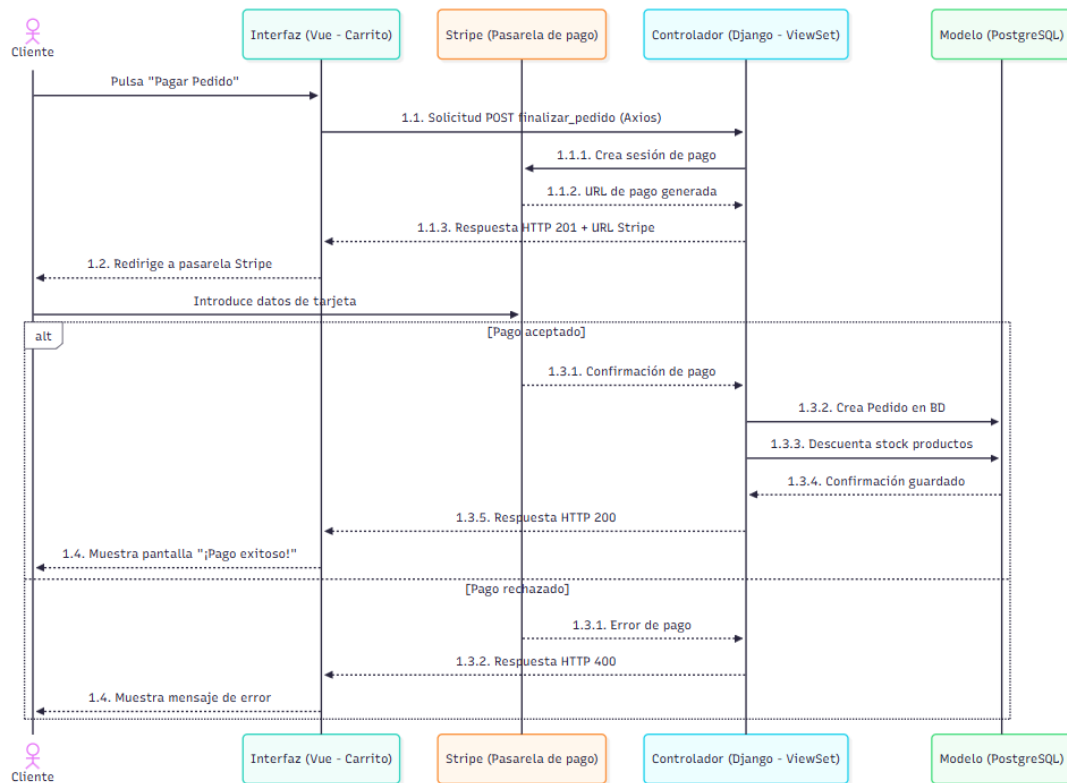


Ilustración 9 Diagrama de secuencia: Pagos

13. Diagrama de paquetes

El diagrama de paquetes muestra la arquitectura general del sistema, dividida en tres capas bien diferenciadas. En la capa superior se encuentra el Paquete Frontend (Vue 3), que gestiona toda la interfaz visual. Internamente, Vue Router se encarga de dirigir al usuario hacia las distintas Vistas y Páginas, que a su vez se apoyan en los Componentes UI reutilizables y en los Servicios/Axios para comunicarse con el servidor.

La comunicación entre el frontend y el backend se realiza mediante Peticiones HTTP a través de la API REST, que actúa como puente entre ambas capas.

En la capa intermedia se encuentra el Paquete Backend (Django), organizado en dos módulos. El Módulo Core contiene los archivos de configuración central (settings.py y urls.py). El Módulo Tienda alberga la lógica de negocio, donde views.py procesa las peticiones, serializers.py traduce los datos a formato JSON y models.py define la estructura de la base de datos.

Finalmente, la capa inferior es el Paquete de Datos, donde PostgreSQL almacena toda la información de forma persistente, accedida desde Django a través de su ORM.

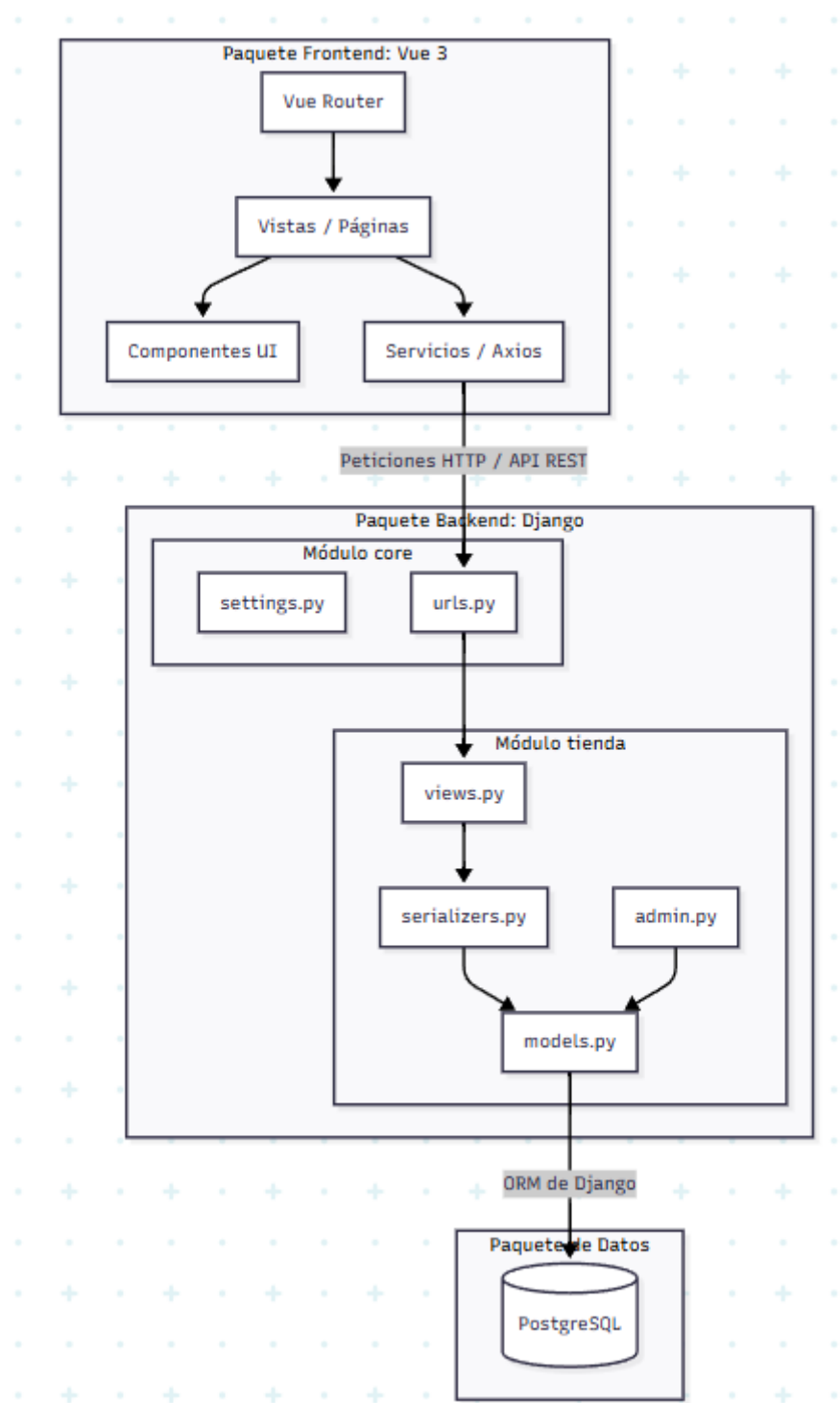


Ilustración 10 Diagrama de paquetes

14. Diagrama de objetos

Este diagrama representa una instancia concreta del sistema, mostrando un ejemplo real de una compra. La usuaria "laura_garcia" realiza un pedido (pedido1) con un total de 53.50€ en estado PAGADO. Este pedido contiene dos líneas de detalle: la primera (detalle1) corresponde a un "Babero Bandana Reversible" del catálogo a 8.50€, clasificado dentro de la categoría "Accesorios Bebé". La segunda línea (detalle2) corresponde a un encargo personalizado (encargo1) de una Mochila Guardería a 45.00€, actualmente en estado EN_PREPARACION, confeccionada con el tejido "Algodón Estrellas" y con el bordado "Sofía".

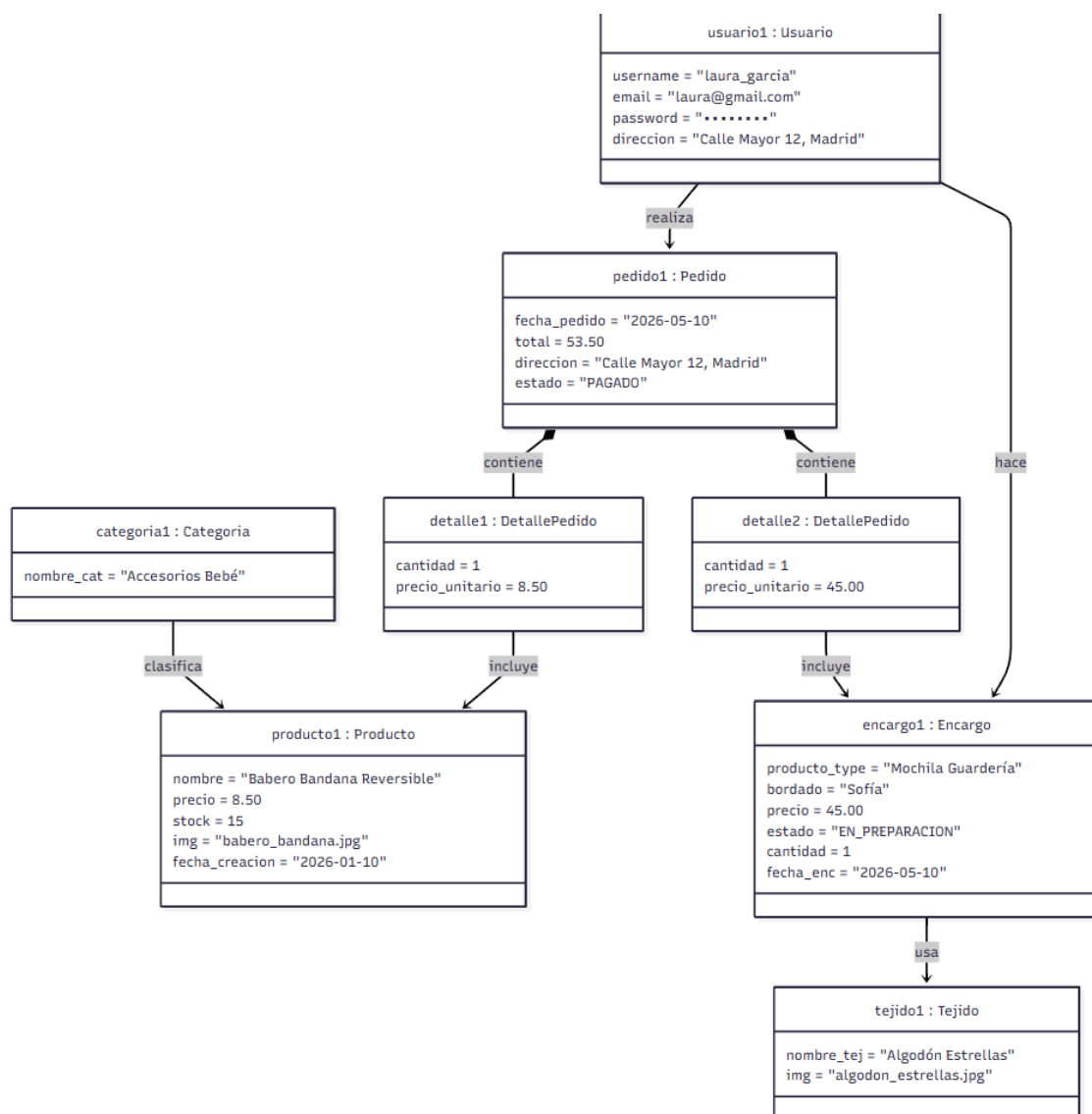


Ilustración 11 Diagrama de objetos

15. Valoraciones

Criterio	Valoración
Funcionalidad implementada	Alta. Se han desarrollado todas las funcionalidades core del sistema: catálogo, encargos personalizados, pasarela de pago, panel de administración y notificaciones por email.
Calidad técnica	Alta. Se ha seguido una arquitectura desacoplada cliente-servidor, aplicando buenas prácticas en la organización del código tanto en el frontend como en el backend.
Usabilidad	Alta. La interfaz es intuitiva y responsive, adaptándose correctamente a dispositivos móviles y de escritorio.
Seguridad	Alta. Se han protegido las rutas privadas mediante autenticación por token JWT y se ha integrado Stripe para evitar el almacenamiento de datos bancarios.
Objetivos alcanzados	Media-Alta. Se han cumplido todos los objetivos planificados a excepción del sistema de favoritos, que queda pendiente para una versión futura.
Viabilidad comercial	Alta. El sistema está en un estado funcional suficiente para su despliegue en producción con ajustes mínimos de infraestructura.

16. Autoevaluación

A lo largo del desarrollo del proyecto se han afrontado retos de distinta naturaleza. El mayor desafío técnico ha sido la gestión del sistema de usuarios y el control de acceso según roles, ya que requería coordinar correctamente la autenticación entre el frontend y el backend, asegurando que cada usuario solo pudiera acceder a las vistas y funcionalidades correspondientes a su perfil. La gestión de pedidos también supuso un

reto, al tener que mantener la coherencia entre los estados del encargo, el pedido y las notificaciones automáticas por correo.

Por otro lado, la integración de la pasarela de pago con Stripe ha sido el aspecto que ha superado las expectativas iniciales. A pesar de la complejidad que suponía gestionar el flujo de confirmación de pago entre Stripe, el backend y el frontend, la implementación resultó más ágil de lo previsto gracias a la documentación oficial de la plataforma y a las herramientas de prueba que proporciona.

En términos generales, se considera que los objetivos definidos al inicio del proyecto se han alcanzado satisfactoriamente, habiendo desarrollado una plataforma funcional y coherente con la visión inicial.

17. Conclusiones

El desarrollo de "Costuras con Mimo" ha permitido consolidar conocimientos en el desarrollo web full-stack, aplicando tecnologías modernas como Vue.js, Django REST Framework y PostgreSQL en un proyecto con un caso de uso real y directo.

El resultado obtenido es una plataforma funcional, con una interfaz cuidada y un backend robusto, que resuelve de forma efectiva el problema inicial: digitalizar y automatizar la gestión de un negocio de artesanía textil que hasta ahora operaba de forma manual a través de redes sociales.

A nivel personal, el proyecto ha puesto de manifiesto la importancia de la constancia y de una documentación rigurosa a lo largo de todo el proceso de desarrollo. Mantener una documentación clara y actualizada no solo facilita la comprensión del código, sino que resulta imprescindible para la escalabilidad y el mantenimiento futuro de cualquier aplicación.

En cuanto a las líneas de trabajo futuro, se identifican las siguientes mejoras prioritarias para una próxima versión de la aplicación:

- Sistema de favoritos: Permitir a los clientes marcar productos de su interés para facilitar futuras compras y obtener datos sobre la demanda del catálogo.
- Personalización del escaparate desde el dashboard: Dotar a la administradora de herramientas para personalizar la apariencia de la tienda pública (colores, imágenes destacadas, banners) sin necesidad de modificar el código.
- Despliegue en producción: Configurar la infraestructura necesaria para que la aplicación sea accesible públicamente a través de un dominio propio.
- Pasarela de email en producción: Sustituir Mailtrap por un servicio de envío de correos real para el entorno de producción.

En definitiva, se considera que el proyecto se encuentra en un estado maduro y suficientemente completo para su lanzamiento al mercado con ajustes mínimos de infraestructura, cumpliendo con el objetivo principal de entregar un producto viable y funcional.

18. Bibliografía/Webgrafía

Vue.js. (2024). *Introduction — Vue.js 3 Official Documentation*. Recuperado el 24 de mayo de 2026, de <https://vuejs.org/guide/introduction.html>

Django REST Framework. (2024). *Django REST Framework — Official Documentation*. Recuperado el 24 de mayo de 2026, de <https://www.django-rest-framework.org/>

Stripe. (2024). *Stripe Developers — Payments Integration Documentation*. Recuperado el 24 de mayo de 2026, de <https://stripe.com/docs/development>

PostgreSQL Global Development Group. (2024). *PostgreSQL Official Documentation*. Recuperado el 24 de mayo de 2026, de <https://www.postgresql.org/docs/>

Vue Router. (2024). *Vue Router — Official Documentation for Single Page Applications*. Recuperado el 24 de mayo de 2026, de <https://router.vuejs.org/>

Mailtrap. (2024). *Mailtrap — Email Testing and Delivery Documentation*. Recuperado el 24 de mayo de 2026, de <https://mailtrap.io/docs/>

pgAdmin Development Team. (2024). *pgAdmin 4 — PostgreSQL Tools Documentation*. Recuperado el 24 de mayo de 2026, de <https://www.pgadmin.org/docs/>

Vite. (2024). *Vite — Next Generation Frontend Tooling Documentation*. Recuperado el 24 de mayo de 2026, de <https://vitejs.dev/guide/>

diagrams.net. (2024). *draw.io — Diagram Software Documentation*. Recuperado el 24 de mayo de 2026, de <https://www.drawio.com/doc/>

Mermaid. (2024). *Mermaid — Diagramming and Charting Tool Documentation*. Recuperado el 24 de mayo de 2026, de <https://mermaid.js.org/intro/>